



THE AMI GROUP

DASHBOARD WEB SYSTEM

Technical Documentation | Version 1.0

PROJECT OVERVIEW

Project ID	Dashboard Web
Version	v1.0.0-node
Organization	The AMI Group
Author	Lê Văn Long

TECH STACK & STATUS

Tech Stack	React + Node/Express + Redis + Doris DB
Workflow Status	Production - Active
Classification	Internal - Confidential

AVAILABILITY

Uptime: 99.9%

Signature: Long

SCALE

500k+ Q/day

Verified: AMI Cloud

PERFORMANCE

Latency: < 100ms

CONFIDENTIAL

Last Update: 2026-08-20 13:43

1. MÔ TẢ TỔNG QUAN HỆ THỐNG

1.1 Giới thiệu

Dashboard Web là hệ thống nội bộ của **The AMI Group** đóng vai trò làm API Gateway và trực quan hóa dữ liệu (Visualization) cho các báo cáo phân tích ratings truyền hình tích hợp với **Apache Superset** và cơ sở dữ liệu phân tích cực nhanh **Doris**.

Hệ thống giải quyết các vấn đề hiệu năng truy cập đồng thời lớn bằng cơ chế bộ nhớ đệm Redis phân cấp, tối ưu hóa hàng đợi kết nối thông qua p-queue, và cung cấp tính năng hủy truy vấn Doris thời gian thực khi người dùng thực hiện ngắt kết nối (kill query). Điều này giúp hệ thống hoạt động ổn định, bảo mật và giảm tải tối đa cho hạ tầng dữ liệu phía sau.

1.2 Vấn đề & Giải pháp

Vấn đề thực tế	Giải pháp của Dashboard Web
Tải dữ liệu báo cáo chậm khi nhiều user cùng truy cập hoặc một user tải nhiều biểu đồ đồng thời.	Redis Caching thông minh: Chuẩn hóa cache key bằng cách loại bỏ user_id và sắp xếp đệ quy các key bộ lọc, tăng tỷ lệ sử dụng lại dữ liệu đệm cũ (cache hit).
Quá tải kết nối đến Apache Superset khi dồn dập các biểu đồ nặng cùng lúc.	Điều phối hàng đợi (P-Queue): Phân chia luồng thành Normal Queue (chạy song song tối đa 50) và Heavy Queue (chạy tuần tự 1-by-1) nhằm bảo vệ dịch vụ đích.
Nghẽn tài nguyên Doris DB do các truy vấn SQL phân tích dữ liệu lớn chạy ngầm quá lâu.	Real-time Query Killing: Tự động tra cứu danh sách tiến trình PROCESSLIST trên Doris DB theo user_id và gửi lệnh KILL QUERY giải phóng tài nguyên lập tức khi user hủy request.
Lỗi hết hạn phiên đăng nhập (session cookie / CSRF token) của API Superset ratings.vtv.vn.	Auto Auth rotation & Retry: Cơ chế kiểm tra, tự động đăng nhập làm mới cookie, tự động gọi lại API (retry) mà không gây gián đoạn trải nghiệm của người dùng.
Khai thác số liệu thô khó khăn cho người dùng không am hiểu kỹ thuật.	Tích hợp Chatbot AI Assistant: Kết nối qua API đến máy chủ LLM chuyên biệt giúp người dùng hỏi đáp dữ liệu tự nhiên.

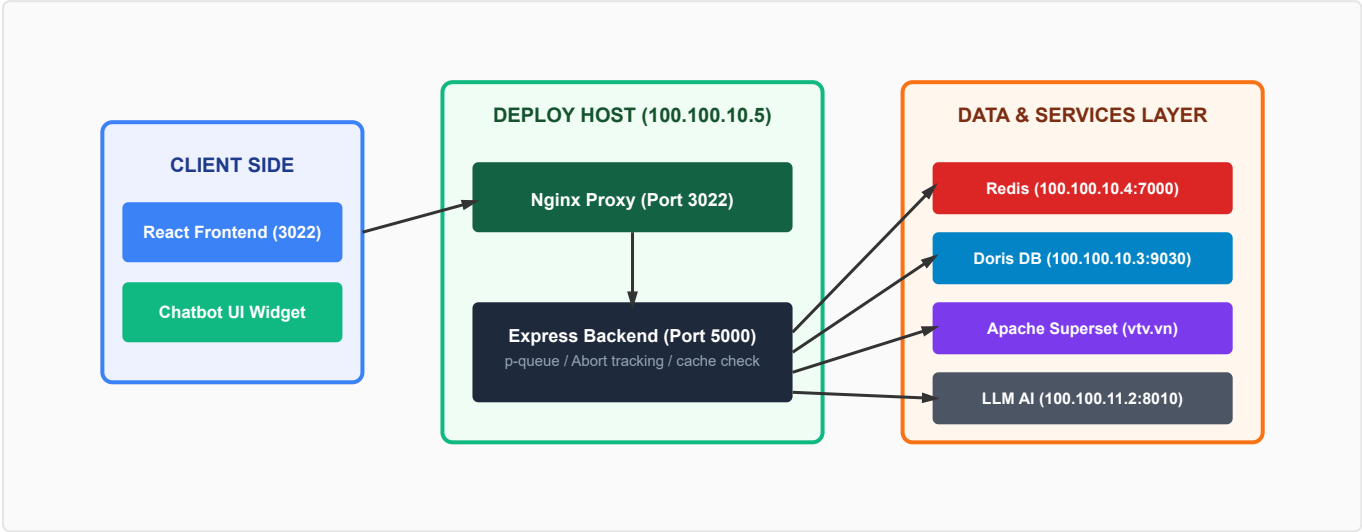
2. MỤC TIÊU SẢN PHẨM

2.1 KPI & Tiêu chí thành công

Chỉ số KPI	Mục tiêu kỹ thuật đạt được
Tỷ lệ cache hit	≥ 70% đối với các truy vấn báo cáo trùng lặp bộ lọc (Province, Target, Kênh...).
Thời gian phản hồi	< 100ms khi dữ liệu có sẵn trong cache; < 5s cho các báo cáo tính toán trực tiếp từ nguồn.
Tỷ lệ kill query Doris	100% các câu lệnh SQL đang chạy trên Doris bị hủy bỏ thành công khi user ngắt kết nối/hủy tải trang.
Thời gian sống của cache	Thiết lập TTL 1 giờ (3600s), đảm bảo cân bằng giữa hiệu năng và độ cập nhật của số liệu rating.
Uptime hệ thống	Uptime ≥ 99.9% hoạt động ổn định trên môi trường sản xuất.

3. KIẾN TRÚC HỆ THỐNG

3.1 Sơ đồ tổng quan kiến trúc kết nối



3.2 Luồng xử lý dữ liệu (Data Flow)

- Yêu cầu dữ liệu:** Client (React) gửi payload truy vấn báo cáo đến API `/api/superset` kèm theo ID người dùng (`user_id`).
- Chuẩn hóa & Kiểm tra Cache:** Backend Express loại bỏ `user_id` khỏi payload, thực hiện deep-sort toàn bộ object và băm SHA-256 tạo cache key. Backend tìm kiếm key trên Redis (100.100.10.4:7000). Nếu tìm thấy, backend trả ngay dữ liệu cho client (Cache Hit).
- Cấp phát Token / Đăng nhập:** Trường hợp Cache Miss, backend kiểm tra tính hợp lệ của token Superset đã lưu trữ. Nếu hết hạn, backend tự động thực hiện đăng nhập đến `https://ratings.vtv.vn/login/`, trích xuất CSRF token mới bằng Cheerio và cập nhật session cookie.
- Điều phối hàng đợi (P-Queue):** Request được đưa vào hàng đợi tương ứng (Normal hoặc Heavy) dựa trên tham số phân loại. Hàng đợi Normal cho phép chạy tối đa 50 luồng đồng thời, trong khi Heavy chạy tuần tự.
- Gọi API gốc và Retry:** Gửi request đến Superset. Nếu bị lỗi token (401/expired), backend xóa cache credentials hiện tại, đăng nhập lại và thực hiện retry.
- Lưu trữ Cache & Trả dữ liệu:** Khi lấy dữ liệu thành công, gộp các luồng dữ liệu (nếu có nhiều query con), lưu vào Redis cache với TTL 3600 giây và trả về client.
- Hủy yêu cầu (Kill logic):** Khi user nhấn hủy truy vấn hoặc đóng tab, Axios Client gửi tín hiệu hủy (AbortController) đến Express Backend. Backend kích hoạt API `/api/kill-user` để ngắt request kết nối tới Superset.
- Hủy query Doris trên DB:** Đồng thời, hệ thống gửi yêu cầu đến `/api/doris/processlist`. Backend kết nối Doris DB (Port 9030), tìm các tiến trình truy vấn tương ứng với `user_id` hiện tại trong bảng `information_schema.PROCESSLIST` và thực thi lệnh `KILL QUERY ''` để giải phóng CPU/RAM của Database.

3.3 CÔNG NGHỆ & TRIỂN KHAI (TECH STACK)

Thành phần	Công nghệ sử dụng	Vai trò chi tiết trong hệ thống
Frontend	React 19 + Vite + Tailwind CSS v4	Xây dựng UI/UX hiển thị biểu đồ trực quan, quản lý trạng thái bộ lọc (Province, Target, Kênh), điều phối hủy request bằng AbortController và tương tác với chatbot.
Backend Gateway	Node.js (v20) + Express	Nhận các yêu cầu từ Frontend, xử lý luồng xác thực tự động với Superset, chuẩn hóa tạo cache key, quản lý hàng đợi và giải phóng tài nguyên Doris DB.
Bộ nhớ đệm (Cache)	Redis (v7)	Lưu trữ kết quả truy vấn API Superset giúp tăng tốc độ tải trang cho các lượt truy cập sau, giảm thiểu request trực tiếp vào API đích.
Cơ sở dữ liệu (DB)	MySQL (Doris DB)	Lưu trữ dữ liệu ratings thô và dữ liệu phân tích; cung cấp bảng information_schema.PROCESSLIST để quản lý truy vấn và hủy bỏ truy vấn.
Hàng đợi (Queue)	p-queue	Kiểm soát tần suất truy cập API, giới hạn số lượng kết nối đồng thời nhằm tránh việc bị rate-limit hoặc làm sập máy chủ Superset.
External APIs	Apache Superset & Chatbot LLM	Superset (ratings.vtv.vn) cung cấp dữ liệu báo cáo chính. LLM API (100.100.11.2:8010) xử lý các câu hỏi ngôn ngữ tự nhiên thành câu trả lời dữ liệu.
Triển khai	Docker & Nginx	Đóng gói mã nguồn thành các Container độc lập. Nginx làm Reverse Proxy định tuyến cuộc gọi, đảm bảo bảo mật và hiệu năng.

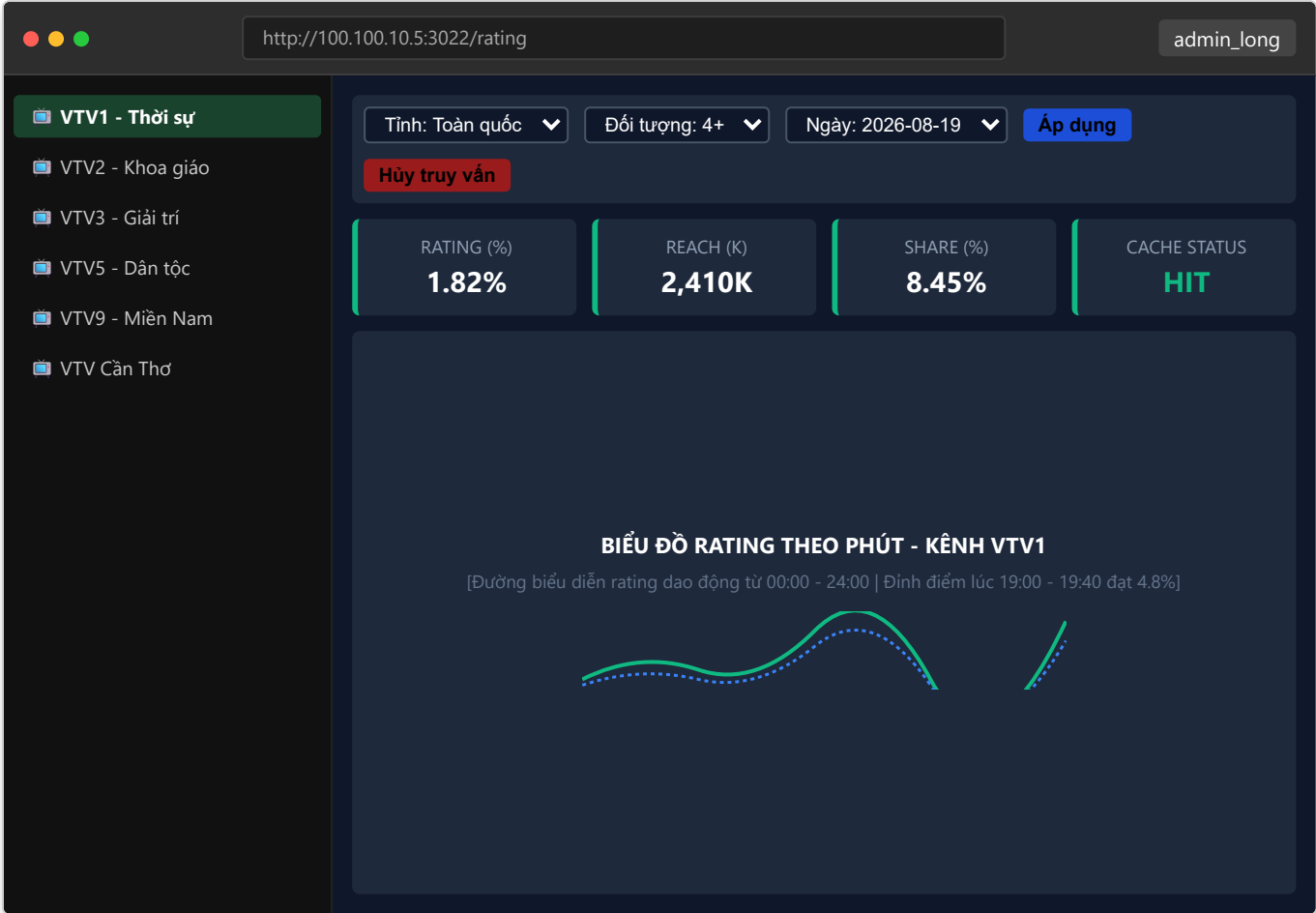
Chi tiết chức năng của các giai đoạn xử lý

- **INPUT (Nguồn thô):** Hệ thống tiếp nhận các tham số lọc do người dùng chọn trên giao diện frontend (kênh, thời gian, tỉnh thành, chỉ số đo lường như rating, reach...).
- **INITIALIZE (Khởi tạo phiên):** Tự động duy trì trạng thái đăng nhập vào hệ thống Apache Superset của VTV. Quá trình lấy cookie và CSRF token được thực hiện ngầm bằng thư viện cheerio để phân tích cú pháp HTML trang đăng nhập.
- **AUTO-PROCESSING (Xử lý tự động):** Cơ chế kiểm tra cache trước khi truy vấn nguồn. Nếu cache bị nhỡ (miss), request sẽ được điều hướng qua hàng đợi p-queue.
- **INTERACTION (Tương tác trực quan):** Người dùng xem báo cáo bằng biểu đồ ECharts/Chart.js có tính tương tác cao (phóng to, thu nhỏ, bật tắt chuỗi dữ liệu). Đồng thời có thể nhập câu hỏi trực tiếp vào khung Chatbot.
- **STORAGE (Lưu trữ đệm):** Dữ liệu trả về được nén và ghi nhận vào Redis với TTL động. Key cache được thiết kế chung cho mọi người dùng có cùng bộ lọc, tăng tối đa tỷ lệ trúng cache.
- **RESOURCE RESOLUTION (Giải phóng tài nguyên):** Khi có thao tác hủy từ người dùng, Express sẽ tự động ngắt kết nối HTTP sang Superset, đồng thời phát lệnh KILL QUERY sang Doris DB dựa trên ID tiến trình đang chạy thuộc sở hữu của user đó, giúp hồi phục CPU/RAM cho máy chủ cơ sở dữ liệu ngay lập tức.

4. GIAO DIỆN HỆ THỐNG

4.1 Màn hình chính & Phân tích Ratings (/rating)

Giao diện Dashboard chính được thiết kế với tông màu tối tối giản, chuyên nghiệp. Phía bên trái là sidebar hiển thị danh sách các kênh truyền hình của VTV (VTV1 - VTV9) và các kênh khu vực. Thanh công cụ ở trên cùng chứa bộ lọc thông minh đa chức năng, nút áp dụng bộ lọc và nút hủy truy vấn nhanh.



Hình: Mô phỏng giao diện chính phân tích ratings truyền hình với trạng thái cache HIT.

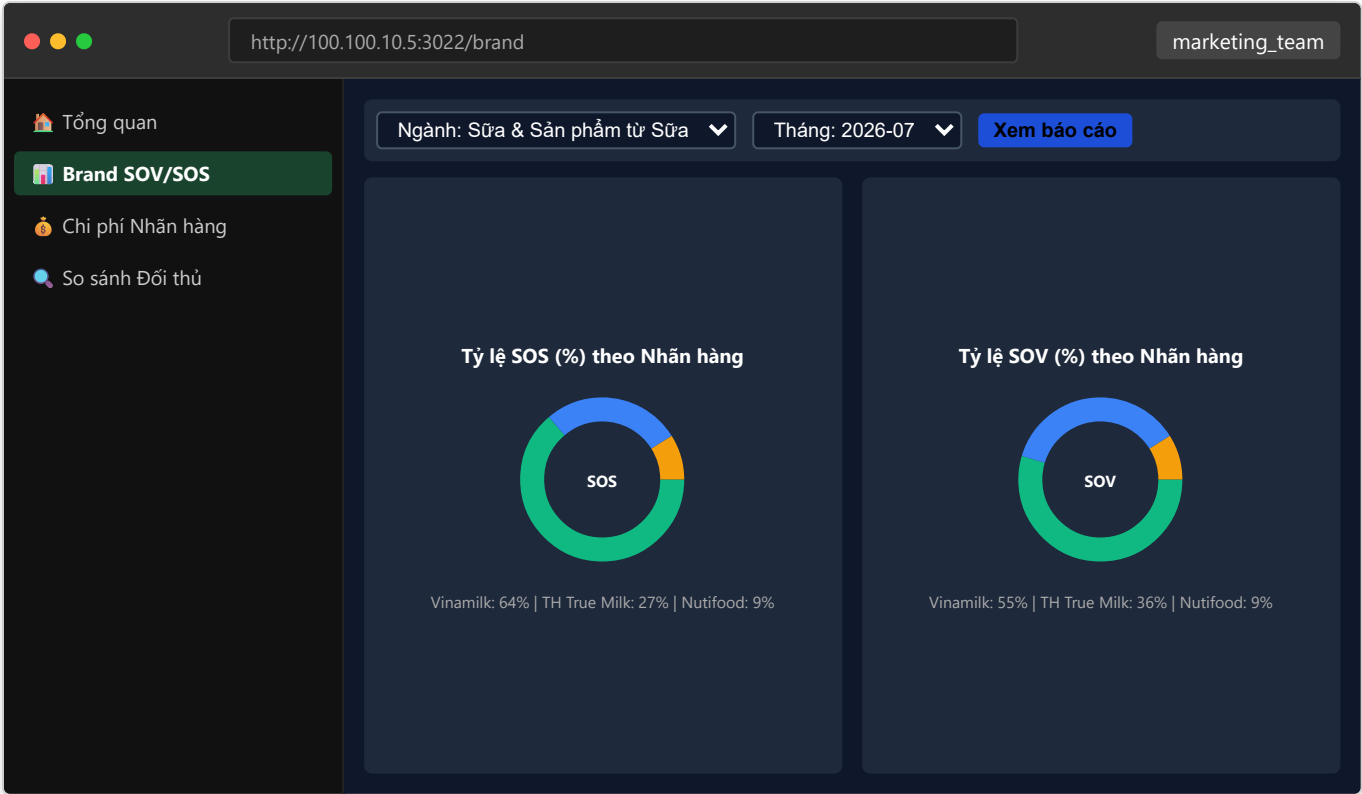
4.2 MÀN HÌNH SPOT ANALYSIS & 4.3 MÀN HÌNH BRAND ANALYSIS

4.2 Màn hình Phân tích Quảng cáo (Spot Analysis)

Màn hình này cho phép đội ngũ kinh doanh theo dõi chi tiết hiệu quả của từng spot quảng cáo phát sóng trên các kênh. Hệ thống tích hợp khả năng tính toán số lượng spot, tổng thời lượng, và ước tính tổng chi phí (Spend VND) theo bảng giá truyền hình hiện hành.

4.3 Màn hình Phân tích Thương hiệu (Brand Analysis)

Hỗ trợ phân tích thị phần quảng cáo của các nhãn hàng lớn và các nhóm sản phẩm cạnh tranh. Trực quan hóa thị phần thông qua các biểu đồ hình tròn (Pie Chart) biểu diễn chỉ số **SOS (Share of Voice - Tỷ lệ thời lượng phát sóng)** và **SOV (Share of Value - Tỷ lệ giá trị quy đổi thành tiền)**.



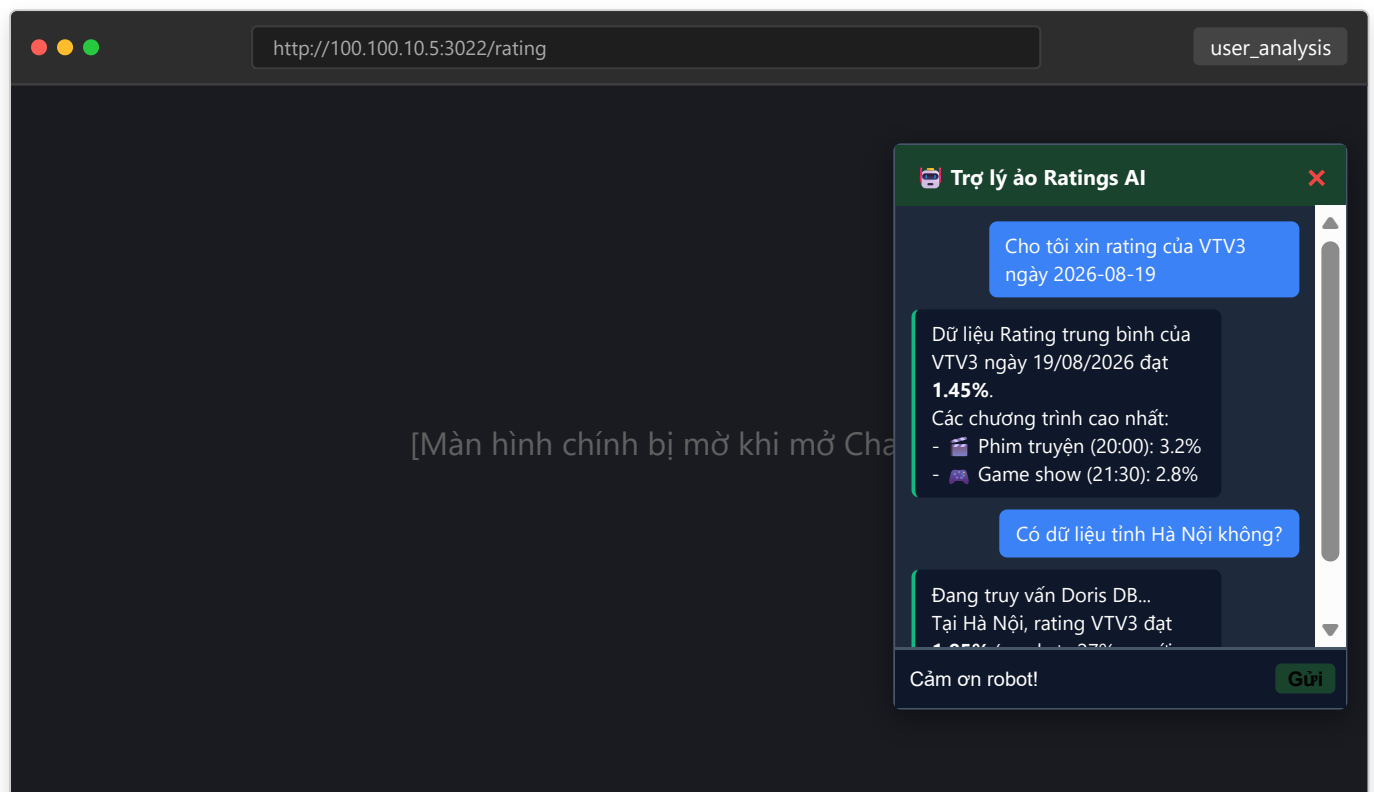
4.4 MÀN HÌNH WORLD CUP 2026 & 4.5 CHATBOT AI ASSISTANT

4.4 Màn hình Báo cáo World Cup 2026

Đây là màn hình chuyên dụng phục vụ các đợt chiến dịch truyền thông lớn, ví dụ như chiến dịch phát sóng trực tiếp **World Cup 2026**. Giao diện tích hợp dữ liệu cập nhật liên tục từ các trận đấu, cung cấp so sánh trực quan về ratings giữa các khung giờ, tỷ lệ tiếp cận khán giả (Reach) và hiệu quả khai thác spot quảng cáo trong các chương trình đồng hành.

4.5 Trợ lý ảo Chatbot AI (Tích hợp thông minh)

Khung Chatbot AI được hiển thị ở góc phải phía dưới của tất cả các trang. Người dùng có thể nhấn vào biểu tượng robot để mở khung chat, thực hiện hỏi đáp nhanh bằng ngôn ngữ tự nhiên. Hệ thống backend Express sẽ chuyển câu hỏi qua API /api/chatbot để giao tiếp với AI Query Engine nội bộ. AI sẽ tự động phân tích ý định, sinh câu lệnh SQL Doris, truy xuất dữ liệu từ database và định dạng câu trả lời dưới dạng bảng số liệu hoặc văn bản hướng dẫn trực quan.



5. BACKEND — NODE.JS / EXPRESS

5.1 Công nghệ & Cấu trúc dịch vụ

Dịch vụ backend được xây dựng trên nền tảng **Node.js** và framework **Express**, đóng vai trò là API Gateway xử lý dữ liệu trung gian và điều phối kết nối. Cấu trúc thư mục tối giản, tập trung xử lý logic ở các file chính:

- `server.js`: File khởi chạy máy chủ Express, định nghĩa các endpoint và kết nối hồ chứa (connection pool) đến Doris DB.
- `api.js`: Lớp wrapper định nghĩa các hàm gọi API đến ratings.vtv.vn thông qua lớp client chuyên dụng.
- `supersetClient.js`: Bộ máy kết nối đến Superset, quản lý vòng đời CSRF token/session cookie và cơ chế phân phối hàng đợi P-Queue (Normal / Heavy).
- `redis.js`: Xử lý logic kết nối Redis Client, xây dựng thuật toán chuẩn hóa cache key thông qua hàm sắp xếp đệ quy (Deep Sort) và lọc mã người dùng.

5.2 Cơ chế hàng đợi & Hủy request (P-Queue, AbortController)

Để ngăn chặn việc trình duyệt gửi dồn dập các request tính toán nặng làm treo dịch vụ Apache Superset hoặc tràn RAM Doris DB, backend đã triển khai cơ chế điều phối hàng đợi và hủy tiến trình ngắt quãng:

- **Queue phân cấp (p-queue)**: Mỗi khi có request, backend kiểm tra payload. Nếu payload có flag `heavy: true`, request sẽ được đẩy vào `heavyQueue` với cấu hình `concurrency: 1` (xử lý tuần tự). Các request nhẹ được đưa vào `normalQueue` với cấu hình `concurrency: 50`.
- **Hủy kết nối HTTP (AbortController)**: Mỗi request gửi sang Superset được gắn kèm một mã signal. Khi client gọi lệnh hủy, backend kích hoạt `signal.abort()` của `AbortController`, ngay lập tức cắt kết nối socket đang tải dữ liệu.
- **Hủy kết nối database Doris**: Express duy trì một connection pool kết nối Doris DB qua driver `mysql2/promise`. Khi nhận lệnh hủy request từ user, backend sẽ thực hiện truy vấn mã `QUERY_ID` đang chạy từ Doris thông qua bảng `information_schema.PROCESSLIST`, sau đó thực thi `KILL QUERY ''` để hạ tải tức thời cho database.

5.3 API Endpoints chính của hệ thống

Method	Endpoint	Mô tả chức năng chi tiết
POST	/api/superset	Endpoint chính tiếp nhận yêu cầu từ React. Kiểm tra cache Redis -> Miss -> Thực thi xếp hàng đợi -> Gọi Superset API -> Gộp dữ liệu -> Trả về & Lưu cache.
POST	/api/doris/processlist	Tra cứu danh sách câu lệnh SQL của <code>user_id</code> đang chạy trên Doris DB và tự động chạy lệnh <code>KILL QUERY</code> giải phóng tài nguyên.
POST	/api/kill-user	Gửi lệnh abort ngắt tất cả các kết nối HTTP đang pending sang Superset của một <code>user_id</code> chỉ định.
POST	/api/chatbot	Nhận câu hỏi từ widget giao diện, proxy cuộc gọi sang server LLM AI (<code>100.100.11.2:8010/api/query</code>).
GET	/api/health	Health Check API phục vụ giám sát hạ tầng (Docker/K8s). Trả về trạng thái hoạt động của server và database.

6. DATABASE & CACHE — DORIS & REDIS

6.1 Cấu trúc kết nối đệm Redis

Hệ thống cache được triển khai trên **Redis v7** nhằm lưu trữ tạm thời các báo cáo nặng có thời gian tính toán lớn.

- **Địa chỉ kết nối:** 100.100.10.4:7000, cơ sở dữ liệu số 0.
- **Thời gian sống (TTL):** Cấu hình mặc định 1 giờ (3600s). Có thể tùy chỉnh thông qua tham số môi trường REDIS_TTL để tối ưu giữa tính cập nhật và hiệu năng.

6.2 Thuật toán chuẩn hóa Cache Key (Cache Key Normalization)

Để đảm bảo hai người dùng khác nhau khi chọn cùng một bộ lọc trên giao diện sẽ nhận chung một kết quả từ bộ nhớ đệm (thay vì tính toán hai lần), backend áp dụng quy trình chuẩn hóa cache key nghiêm ngặt:

```
Hàm buildCacheKey(payload):
1. Loại bỏ tham số "user_id" ở cấp cao nhất của payload.
2. Quét qua mảng payload.queries và lọc bỏ các filter có col = "user_id".
3. Chạy hàm sortDeep(payload_cleaned):
   - Nếu là Array: lặp lại sortDeep cho từng phần tử.
   - Nếu là Object: sắp xếp thứ tự các Key theo bảng chữ cái A-Z,
     sau đó gọi đệ quy sortDeep cho từng Value.
4. Chuyển kết quả đã sắp xếp thành chuỗi JSON chuẩn hóa (JSON.stringify).
5. Băm chuỗi JSON bằng thuật toán SHA-256 thu được chuỗi Hex dài 64 ký tự.
6. Trả về Cache Key dưới định dạng: "web_<SHA256_HEX>".
```

6.3 Cơ chế tương tác Doris PROCESSLIST & Kill query

Khi người dùng gửi truy vấn báo cáo, câu lệnh SQL sẽ được thực thi trên Doris DB. Để quản lý và dọn dẹp các truy vấn mồ côi (khi client đã ngắt kết nối nhưng DB vẫn chạy), hệ thống triển khai cơ chế giám sát tiến trình:

- **Liên kết tiến trình:** Dữ liệu payload khi gọi sang DB luôn được gắn kèm chú thích chứa ID người dùng (ví dụ: /* user_id: admin_long */ SELECT ...).
- **Truy vấn PROCESSLIST:** Khi nhận tín hiệu hủy, Express kết nối qua tài khoản Ami_doris đến Doris DB, thực thi câu lệnh SQL để quét các câu lệnh đang chạy chứa mã định danh người dùng:

```
SELECT QUERY_ID, Info
FROM information_schema.PROCESSLIST
WHERE COMMAND = 'Query' AND Info LIKE '%admin_long%';
```

- **Kích hoạt lệnh KILL:** Với mỗi kết quả trả về, backend trích xuất QUERY_ID và chạy lệnh ngắt tiến trình:

```
KILL QUERY '<QUERY_ID>';
```

Quá trình này giải phóng CPU/RAM trên cụm Doris DB ngay lập tức, ngăn ngừa tình trạng thắt nút cổ chai (bottleneck) hệ thống.

7. TRIỂN KHAI & CẤU HÌNH (DEPLOYMENT & CONFIGURATION)

7.1 Thông số môi trường triển khai

Thành phần dịch vụ	Thông số IP & Cổng kết nối	Môi trường hoạt động
Server triển khai	VM Cloud	Môi trường ảo hóa nội bộ nhóm AMI.
Địa chỉ IP chính	100.100.10.5	Địa chỉ IP đích phục vụ người dùng.
Cổng Frontend (React)	Port 3022	Được serve tĩnh thông qua Nginx bên trong Docker container.
Cổng Backend (Express)	Port 5000	Chạy trực tiếp Node.js Express server trong Docker container.
Redis Cache	100.100.10.4:7000	Cụm Redis độc lập phục vụ lưu trữ cache đệm.
Doris DB (MySQL)	100.100.10.3:9030	Cơ sở dữ liệu lưu trữ ratings thô phục vụ phân tích.
Chatbot LLM API	100.100.11.2:8010	Server LLM nội bộ chịu trách nhiệm trả lời chatbot.
API đích Superset	https://ratings.vtv.vn	Nguồn cấp dữ liệu ratings gốc của nhà đài.

7.2 Chi tiết file cấu hình môi trường (.env)

Cấu hình Backend (/backend_superset/.env)

```
# Superset Config
SUPERSET_URL=https://ratings.vtv.vn
SUPERSET_USERNAME=Long
SUPERSET_PASSWORD=Longouttrinh

# Server Config
PORT=5000
NODE_ENV=production

# Redis Config
REDIS_HOST=100.100.10.4
REDIS_PORT=7000
REDIS_TTL=3600
REDIS_DB=0
```

Cấu hình Frontend (/frontend/.env)

```
# API Connection
VITE_API_BASE_URL=http://100.100.10.5:5000
VITE_API_ROUTE=/api/superset
VITE_API_TIMEOUT=300000

# Metadata
VITE_API_DOMAIN=https://ratings.vtv.vn
```

7.3 Hướng dẫn vận hành hệ thống hàng ngày

Hệ thống hoạt động hoàn toàn tự động. Tuy nhiên nhân viên vận hành cần chú ý kiểm tra giám sát:

- **Theo dõi Logs Nginx và Docker:** Đảm bảo hai container frontend và backend luôn ở trạng thái Up. Kiểm tra bằng lệnh docker compose ps.
- **Kiểm tra trạng thái Cache:** Khi biểu đồ tải tức thì dưới 100ms và hiển thị nhãn "from_cache: true", tức là hệ thống cache đang hoạt động tối ưu. Trường hợp dữ liệu bị lệch so với dữ liệu broadcast, có thể thực hiện xóa cache Redis.
- **Quản lý tiến trình Doris:** Thường xuyên kiểm tra danh sách tiến trình mờ còi trên Doris DB bằng cách kết nối vào cổng 9030 và chạy lệnh SHOW PROCESSLIST; để giám sát tải của hệ thống.

--- HẾT TÀI LIỆU KỸ THUẬT ---